COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 | **SECTION:** | CPE21S1 |
| **DATE PERFORMED:** | July 23, 2026 | **DATE SUBMITTED:** | July 27, 2026 |
| **NAME:** | Al Twain D. Pasion | **INSTRUCTOR:** | Engr Quejado |

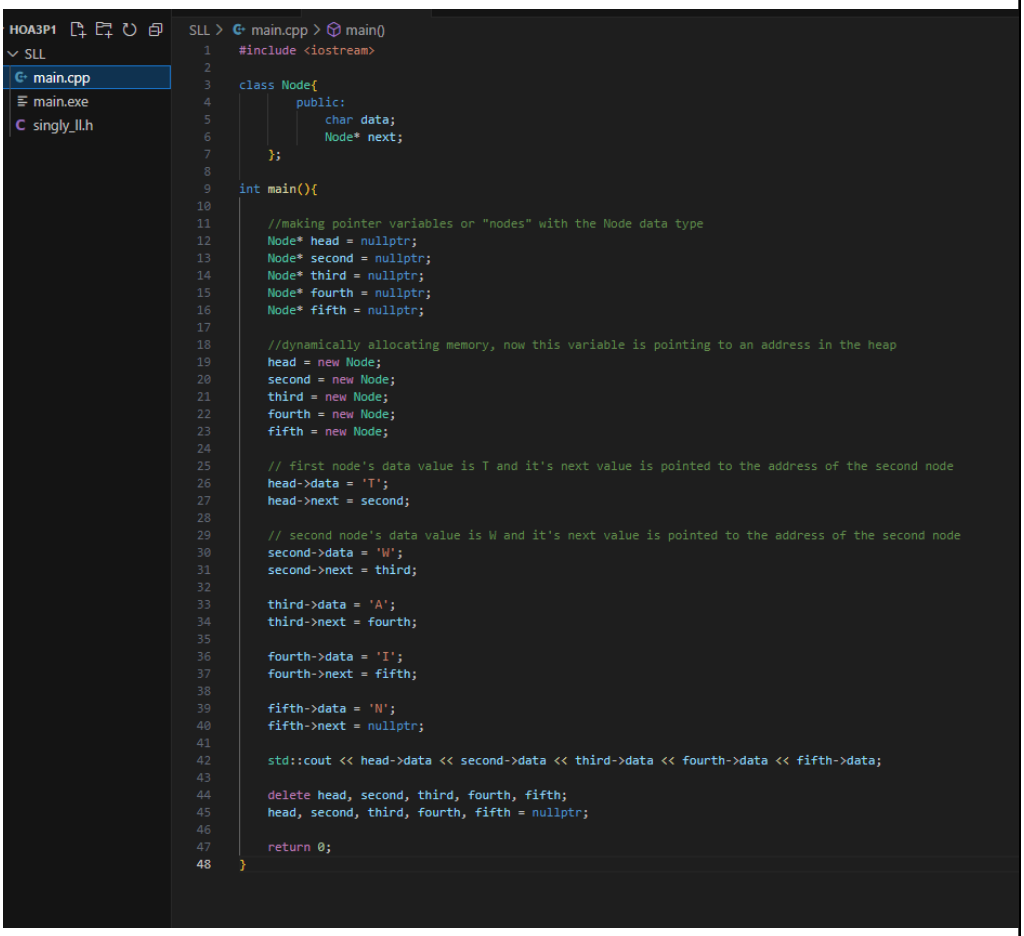
## ACTIVITY NO. 3 Linked List

## 1. PROCEDURE OUTPUT

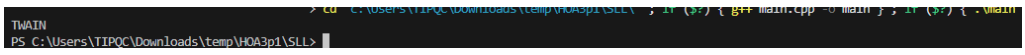
What procedures did you perform?

*Table 3-1 Output of Initial/Simple Implementation*

**Screenshot**

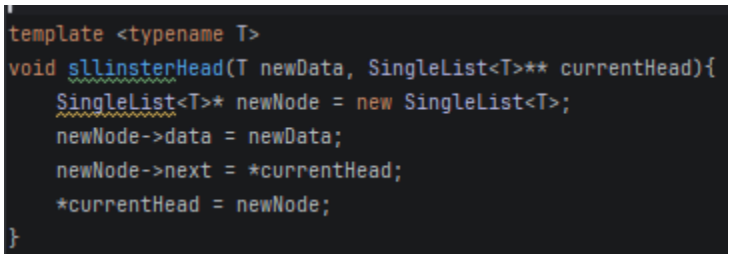
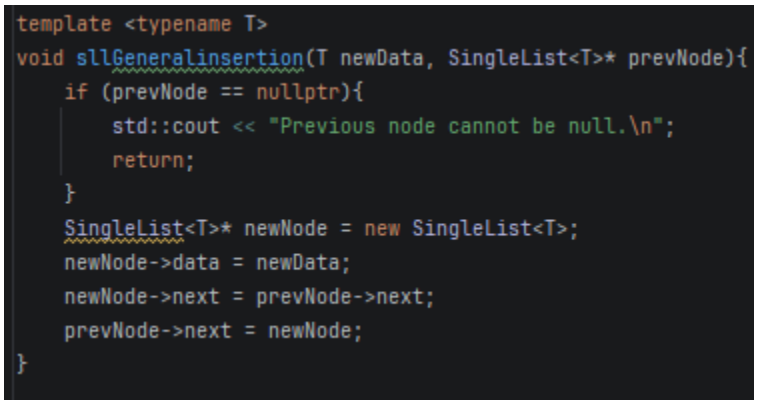



**Output**



| DISCUSSION |
| --- |
| **The program starts with introducing a new class “Node” with its public members being a character value “data” and pointer value “next”. After that we assign new variables to this class, the first one being head and the rest are second, third, etc. We then “link” these variables by assigning their pointer values next to the next node. This then makes them into a list, when printed out, it’ll print the data starting from head to the end of the list. We can make improvements on how to initialize the values and how we print the data. It’s a bit redundant and repetitive.** |

*Table 3-2 Code for the List Operations*

| Operation | Screenshot |
| --- | --- |
| Traversal |  |
| Insertion at Head |  |
| Insert at any part of the list |  |

```
template <typename T>
void listTraversal(SingleList<T>* head){
    while (head != nullptr){
        std::cout << head->data;
        if (head->next != nullptr){
            std::cout << "->";
        }
        head = head->next;
    }
    std::cout << std::endl;
}
```

```
template <typename T>
void sllinsterHead(T newData, SingleList<T>** currentHead){
    SingleList<T>* newNode = new SingleList<T>;
    newNode->data = newData;
    newNode->next = *currentHead;
    *currentHead = newNode;
}
```

```
template <typename T>
void sllGeneralinsertion(T newData, SingleList<T>* prevNode){
    if (prevNode == nullptr){
        std::cout << "Previous node cannot be null.\n";
        return;
    }
    SingleList<T>* newNode = new SingleList<T>;
    newNode->data = newData;
    newNode->next = prevNode->next;
    prevNode->next = newNode;
}
```

Insertion at end

```
template<typename T>
void sllinsertEnd(T newData, SingleList<T>** head){
    SingleList<T>* newNode = new SingleList<T>;

    SingleList<T>* currentNode = *head;

    newNode->data = newData;
    newNode->next = nullptr;

    if (currentNode == nullptr){
        currentNode = newNode;
        return;
    }
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }
    currentNode->next = newNode;
}
```

Deletion of a node

```
template<typename T>
void sllDelete(T findData, SingleList<T>** head){
    if(*head == nullptr){
        return;
    }
    SingleList<T>* currentNode = *head;
    SingleList<T>* prevNode = nullptr;

    while(currentNode != nullptr && currentNode->data != findData){
        prevNode = currentNode;
        currentNode = currentNode->next;
    }
    if(currentNode == nullptr){
        return;
    }
    if (prevNode == nullptr){
        *head = currentNode->next;
    }
    else{
        prevNode->next = currentNode->next;
    }
    delete currentNode;
}
```

*Table 3-3 Code and Analysis for Singly List*

<table>
<tr><td rowspan="2">a.</td><td>Source Code</td><td>

```
template <typename T>
void listTraversal(SingleList<T>* head){
    while (head != nullptr){
        std::cout << head->data;
        if (head->next != nullptr){
            std::cout << "";
        }
        head = head->next;
    }
    std::cout << std::endl;
}
```

```
std::cout << "Linked List Traversal:\n";
listTraversal(head);
```

</td></tr>
<tr><td>Console</td><td>

```
Linked List Traversal:
CPE101
```

</td></tr>
<tr><td rowspan="2">b.</td><td>Source Code</td><td>

```
template <typename T>
void sllinsterHead(T newData, SingleList<T>** currentHead){
    SingleList<T>* newNode = new SingleList<T>;
    newNode->data = newData;
    newNode->next = *currentHead;
    *currentHead = newNode;
}
```

```
std::cout << "New Head Insertion:\n";
sllinsterHead( newData: 'G',&head);
listTraversal(head);
```

</td></tr>
<tr><td>Console</td><td>

```
New Head Insertion:
GCPE101
```

</td></tr>
<tr><td>c.</td><td>Source Code</td><td>

```
template <typename T>
void sllGeneralinsertion(T newData, SingleList<T>* prevNode){
    if (prevNode == nullptr){
        std::cout << "Previous node cannot be null.\n";
        return;
    }
    SingleList<T>* newNode = new SingleList<T>;
    newNode->data = newData;
    newNode->next = prevNode->next;
    prevNode->next = newNode;
}
```

</td></tr>
</table>

<table>
<tr><td></td><td></td><td>

```
std::cout << "General Insertion:\n";
sllGeneralinsertion( newData: 'E', second->next);
listTraversal(head);
```

</td></tr>
<tr><td></td><td>Console</td><td>

```
General Insertion:
GCPEE101
```

</td></tr>
<tr><td>d.</td><td>Source Code</td><td>

```
template<typename T>
void sllDelete(T findData, SingleList<T>** head){
    if(*head == nullptr){
        return;
    }
    SingleList<T>* currentNode = *head;
    SingleList<T>* prevNode = nullptr;

    while(currentNode != nullptr && currentNode->data != findData){
        prevNode = currentNode;
        currentNode = currentNode->next;
    }
    if(currentNode == nullptr){
        return;
    }
    if (prevNode == nullptr){
        *head = currentNode->next;
    }
    else{
        prevNode->next = currentNode->next;
    }
    delete currentNode;
}
```

```
std::cout << "Deleting.. \n";
sllDelete( findData: 'C', &head);
listTraversal(head);
```

</td></tr>
<tr><td></td><td>Console</td><td>

```
Deleting..
GPEE101
```

</td></tr>
</table>

e.

**Source Code**

```
template<typename T>
void sllDelete(T findData, SingleList<T>** head){
    if(*head == nullptr){
        return;
    }
    SingleList<T>* currentNode = *head;
    SingleList<T>* prevNode = nullptr;

    while(currentNode != nullptr && currentNode->data != findData){
        prevNode = currentNode;
        currentNode = currentNode->next;
    }
    if(currentNode == nullptr){
        return;
    }
    if (prevNode == nullptr){
        *head = currentNode->next;
    }
    else{
        prevNode->next = currentNode->next;
    }
    delete currentNode;
}
```

```
std::cout << "Deleting.. \n";
sllDelete( findData: 'P', &head);
listTraversal(head);
```

**Console**

```
Deleting..
GEE101
```

f.

**Source Code**

```
template <typename T>
void listTraversal(SingleList<T>* head){
    while (head != nullptr){
        std::cout << head->data;
        if (head->next != nullptr){
            std::cout << "";
        }
        head = head->next;
    }
    std::cout << std::endl;
}
```

```
listTraversal(head);
```

**Console**

```
GEE101
```

**ANALYSIS:**

**a.**
**This is a function that lets us move through a singly list sequentially. We have a while loop where the condition is, if the head is not a null pointer, it will continue the loop and print the node's data. Also in this while loop, we have a condition that if the next node is not a null pointer it print out a space. Lastly, it'll assign the current head's next value as the new head temporarily so that we can go forward. This will loop until we've reached the end of the list. Now we printed out CPE101, which is the list from head to last node.**

**b.**
**This is a function that lets us create a new head in a singly list. It first creates a new node in the heap and assigns its data and next value to the new data assigned and the current head respectively. Lastly, it'll assign the new node as the new head of the list by dereferencing the current head. After printing, we can see it has now turned from CPE101, to GCPE101 with the G being the new head.**

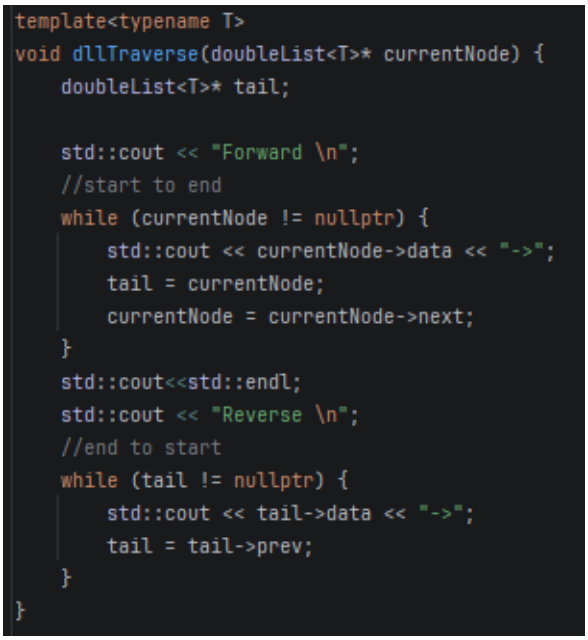
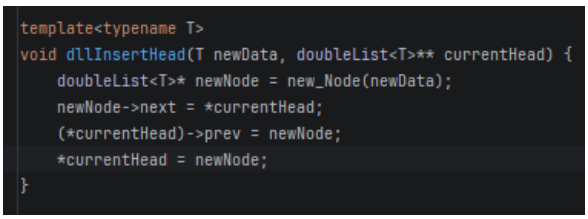
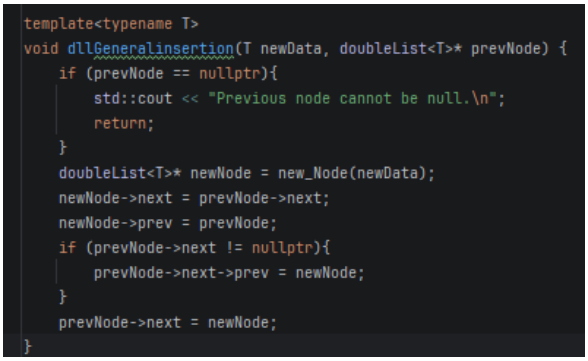
**c.**
**This is a function that lets us create a new node anywhere in a singly list after the chosen node. It first checks if the chosen node is a null pointer, if true, it will end the function immediately. If it is false, then it will make a new node, assign the new data to this node and its next value to the chosen node's next value. Lastly, it'll put the chosen node's next value and assign the new newly created node to it, this puts it in-front of the chosen node. For this example, we choose the second's next value which is the data that has an E. After printing we placed it after the first E of the list and it became GCPEE101.**

**d.**
**The delete function will remove a node and fix up the loose pointers in a singly list. First, it will check if the head pointer value is a null pointer, if true, it will end the function. If not, it'll continue with the function. Now we assign a temporary variable for our current node (the head) and the previous node before it, this will help us traverse the list looking for our desired data to delete and clean up afterwards. It has two conditions, if the current node is not a null pointer AND it is not the data, we keep traversing the list. If we ever hit the end of the list without finding the data, it will just end the function. Now with the previous node variable we just initialized earlier, we can use this to reconnect the desired data's next and previous value together so that it doesn't create loose pointers. After this, we delete the node. When we run this function with the desired data being C, we can see that we effectively deleted the letter C and it now is GPEE101.**

**e.**
**Now we wanted to delete P, after running the delete function. We've repeated the process given in letter D and removed P.**

**f.**
**Now this is our final printed singly list after all the changes we have done to it. From CPE101 to GEE101.**

*Table 3-4 Modified Operations For Doubly Linked List*

| ***Screenshot(s)*** | ***Analysis*** |
|---|---|
|  <br>```<br>template<typename T><br>void dllTraverse(doubleList<T>* currentNode) {<br>    doubleList<T>* tail;<br><br>    std::cout << "Forward \n";<br>    //start to end<br>    while (currentNode != nullptr) {<br>        std::cout << currentNode->data << "->";<br>        tail = currentNode;<br>        currentNode = currentNode->next;<br>    }<br>    std::cout<<std::endl;<br>    std::cout << "Reverse \n";<br>    //end to start<br>    while (tail != nullptr) {<br>        std::cout << tail->data << "->";<br>        tail = tail->prev;<br>    }<br>}<br>``` | The traversal for a doubly linked list is fundamentally the same, but with an added “previous” pointer. For the first loop, it prints every data from the head to the last, this is while it’s saving the current node as a “tail” or a checkpoint to go back to for the second loop. Now this second loop uses that checkpoint as a starting point to print “in-reverse”, so from the last to the head again. |
|  <br>```<br>template<typename T><br>void dllInsertHead(T newData, doubleList<T>** currentHead) {<br>    doubleList<T>* newNode = new_Node(newData);<br>    newNode->next = *currentHead;<br>    (*currentHead)->prev = newNode;<br>    *currentHead = newNode;<br>}<br>``` | The insert head for a doubly linked list is quite similar to the insert head for a singly linked list. The difference is just the added “prev” pointer. So we’re adding a new node and setting its “next” pointer to the current Head of the list and also setting the current Head’s “prev” pointing to the newly added nod, this effectively added the new nod at the very first of the list. Now we set the “head” variable as the new node. |
|  <br>```<br>template<typename T><br>void dllGeneralinsertion(T newData, doubleList<T>* prevNode) {<br>    if (prevNode == nullptr){<br>        std::cout << "Previous node cannot be null.\n";<br>        return;<br>    }<br>    doubleList<T>* newNode = new_Node(newData);<br>    newNode->next = prevNode->next;<br>    newNode->prev = prevNode;<br>    if (prevNode->next != nullptr){<br>        prevNode->next->prev = newNode;<br>    }<br>    prevNode->next = newNode;<br>}<br>``` | I did some modifications of the singly linked list’s general insertion to make it work for the doubly linked list. So firstly, it protects itself from receiving a nullptr. After that, it initializes a new node utilizing the new_Node function. After that it sets up the new Node’s “next” to be the previous Node’s “next”, and also sets up the new Node’s “prev” as the previous node. One more final step is to check if the previous Node was initially pointing to a null pointer, if it wasn’t we can assign the node after the previous Node’s “prev” to be the new Node. And then we finally set the previous Node’s “next” to be the new Node. |

```
template<typename T>
void dllInsertTail(T newData, doubleList<T>* currentHead) {
    doubleList<T>* newNode = new_Node(newData);

    while (currentHead->next != nullptr) {
        currentHead = currentHead->next;
    }
    currentHead->next = newNode;
    newNode->prev = currentHead;
}
```

So the Insert tail for the doubly linked list is pretty simple and short. First it initializes a new Node, then it uses a while loop to go at the very end of the list and assigning the node as the “current Head”. Now it sets the “current Head”’s “next” value as the new Node and the new Node’s prev as the “current Head”. This then successfully added the new Node at the last of the list

```
template<typename T>
void dllDelete(T findData, doubleList<T>** head) {
    if (*head == nullptr) {
        return;
    }
    doubleList<T>* currentNode = *head;
    doubleList<T>* prevNode = nullptr;

    while(currentNode != nullptr && currentNode->data != findData){
        prevNode = currentNode;
        currentNode = currentNode->next;
    }
    if(currentNode == nullptr){
        return;
    }
    if (prevNode == nullptr){
        *head = currentNode->next;
        if (*head != nullptr) {
            (*head)->prev = nullptr;
        }
    }
    else{
        prevNode->next = currentNode->next;
        if (currentNode->next != nullptr){
            currentNode->next->prev = prevNode;
        }
        delete currentNode;
    }
}
```

The delete function has the same concept as the singly linked list when finding the data, it’s the deleting part that’s different. If I were to explain it line by line starting from there, the “if” condition is if we didn’t find the data, it would just return. The second would be if the data we’ve found has its previous pointer value pointing to none, it’ll assign the next pointer value as the new head and a nested “if” will check if this new head is a null pointer, if it’s not it will set its previous pointer to null pointer. But when the condition is not met, it’ll run the “else”. This makes sure that we patch up the connection between the next node and the previous node of the current node. This greatly sets up the delete syntax.

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

**ILO B: Solve given problems utilizing linked lists in C++**

**Problem Title**: Implementing a Song Playlist using Linked List
**Source**: Packet Publishing

Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

## SONG PLAYLIST HEADER FILE

```cpp
#ifndef SONGPLAYLIST_H
#define SONGPLAYLIST_H

template<typename baka>
class playlist {
public:
    baka song;
    baka artist;
    playlist<baka>* next = nullptr;
    playlist<baka>* prev = nullptr;
};

// default constructor for creating songs
template<typename baka>
playlist<baka>* newSong (baka title, baka author) {
    playlist<baka>* new_Song = new playlist<baka>;
    new_Song->song = title;
    new_Song->artist = author;
    new_Song->next = nullptr;
    new_Song->prev = nullptr;
    return new_Song;
}

// printing the whole playlist
template<typename baka>
void songList (playlist<baka>* firstS) {
    if (firstS == nullptr) {
        std::cout << "Playlist is empty.\n\n";
        return;
    }

    std::cout << "<-------------Start of Playlist------------->\n";
    int count = 1;
    playlist<baka>* firstSong = firstS;
    std::cout << count << ". " << firstS->song << " - " << firstS->artist << std::endl;
    firstS = firstS->next;
    while (firstS != firstSong) {
        count += 1;
        std::cout << count << ". " << firstS->song << " - " << firstS->artist << std::endl;
        firstS = firstS->next;
    }
    std::cout << "<-------------End of Playlist------------->\n";

}
```

```cpp
// adding new song, it'll default to the very back
template<typename baka>
void addSong (std::string title, std::string author, playlist<baka>* firstS ) {
    playlist<baka>* prevSong = firstS->prev;

    playlist<baka>* music = newSong( title,  author);
    music->prev = prevSong;
    music->next = firstS;
    firstS->prev = music;
    prevSong->next = music;

    std::cout << music->song << " added.\n";

}

//deleting a song
template<typename baka>
void deleteSong (std::string title, playlist<baka>** head, playlist<std::string>** playing) {
    playlist<baka>* currentSong = *head;
    playlist<baka>* firstSong = *head;

    while (currentSong->song != title) {
        currentSong = currentSong->next;
        if (currentSong == firstSong) {
            std::cout << "Song not found... \n";
            return;
        }
    }

    playlist<baka>* nextSong = currentSong->next;
    playlist<baka>* prevSong = currentSong->prev;

    nextSong->prev = prevSong;
    prevSong->next = nextSong;

    // checks if the song about to be deleted is the first song
    if (currentSong == *head) {
        // checks if this song points to itself, meaning it is the only song in the list
        if (currentSong == nextSong) {
            // turns the list into a null
            *head = nullptr;
        } else {
            // assigns the next song in the list to be the first song so from 2nd to 1st
            *head = nextSong;
        }
    }
```

```
92
93          std::cout << "\n '" << title << " by " << currentSong->artist << " ' deleted.\n\n";
94          if (currentSong == *playing) {
95              *playing = nullptr;
96          }
97          delete currentSong;
98      }
99
100     // playing a song
101     template<typename baka>
102     void playSong (std::string title, playlist<baka>** playing, playlist<baka>* head) {
103         playlist<baka>* currentSong = head;
104         playlist<baka>* firstSong = head;
105
106         while (currentSong->song != title) {
107             currentSong = currentSong->next;
108             if (currentSong == firstSong) {
109                 std::cout << "Song not found... \n";
110                 return;
111             }
112         }
113
114         *playing = currentSong;
115         std::cout << "\nNow playing: " << currentSong->song << " - " << currentSong->artist << "\n";
116     }
117
118     // moving to the next song
119     template<typename baka>
120     void nextSong (playlist<baka>** playing) {
121         if (*playing == nullptr) {
122             std::cout << "\nNothing is playing...\n";
123             return;
124         }
125         *playing = (*playing)->next;
126         std::cout << "\nNow playing: " << (*playing)->song << " - " << (*playing)->artist << "\n";
127     }
128
129     // going back to the previous song
130     template<typename baka>
131     void prevSong (playlist<baka>** playing) {
132         if (*playing == nullptr) {
133             std::cout << "Nothing is playing...\n";
134             return;
135         }
```

```
136         *playing = (*playing)->prev;
137         std::cout << "\nNow playing: " << (*playing)->song << " - " << (*playing)->artist << "\n";
138     }
139
140     // deleting entire playlist
141     template<typename baka>
142     void deleteplaylist (playlist<baka>** head, playlist<baka>** playing) {
143         if (*head == nullptr) {
144             std::cout << "Playlist is empty.\n";
145             return;
146         }
147
148         playlist<baka>* currentSong = *head;
149         playlist<baka>* firstSong = *head;
150
151         do {
152             playlist<baka>* temp = currentSong;
153             currentSong = currentSong->next;
154             delete temp;
155         } while (currentSong != firstSong);
156
157         *head = nullptr;
158         *playing = nullptr;
159
160         std::cout << "\nPlaylist successfully deleted.\n";
161     }
162
163     // pause the song
164     template<typename baka>
165     void pauseSong (playlist<baka>** playing, bool* isPaused) {
166         if (*playing == nullptr) {
167             std::cout << "Nothing is playing...\n";
168             return;
169         }
170         if (*isPaused) {
171             std::cout << "Song is already paused\n";
172         } else {
173             std::cout << (*playing)->song << " - " << (*playing)->artist << " is now paused...\n";
174             *isPaused = true;
175         }
176     }
177
```

```cpp
// resume the song
template<typename baka>
void resumeSong (playlist<baka>** playing, bool* isPaused) {
    if (*playing == nullptr) {
        std::cout << "Nothing is playing...\n";
        return;
    }
    if (*isPaused == false) {
        std::cout << "Song is already playing\n";
    } else {
        std::cout << (*playing)->song << " - " << (*playing)->artist << " is now resumed\n";
        *isPaused = false;
    }
}

#endif #ifndef SONGPLAYLIST_H
```

## MAIN PROGRAM FILE

```cpp
#include <iostream>
#include "songPlaylist.h"
#include <string>


int main() {
    playlist<std::string>* head = nullptr;
    playlist<std::string>* playing = nullptr;
    bool isPaused = false;

    // showing that the playlist empty by default
    std::cout << "--- Default State ---\n";
    songList(head);

    // gonna add the first song and have the head pointer value pointed to that
    head = newSong<std::string>( title: "New Genesis", author: "Ado");
    head->next = head;
    head->prev = head;

    // now we add more songs
    addSong( title: "Caramel Pain", author: "Hoshimachi Suisei", head);
    addSong( title: "Sparkling Daydream", author: "ZAQ", head);
    addSong( title: "Trapped in the Past", author: "Tuyu", head);
    addSong( title: "If I Could Be A Constellation", author: "kessoku band", head);

    std::cout << "\n--- After adding songs ---\n";
    songList(head);

    // now we'll play a song
    playSong( title: "Sparkling Daydream", &playing, head);

    // now we're gonna use the next and previous functions
    std::cout << "\n--- Spamming Next Button ---\n";
    nextSong(&playing); // should move to "Trapped in the Past"
    nextSong(&playing); // should be at "kessoku band"
    nextSong(&playing); // should be at "Ado"
    std::cout << "\n--- Clicking the Previous Button ---\n";
    prevSong(&playing);      // back to "kessoku band"

    // demonstrating the pause and resume and the significance of the bool variable
    std::cout << "\n--- Pause and Resuming---\n";
    pauseSong(&playing, &isPaused);
    pauseSong(&playing, &isPaused);   // should say "already paused"
    resumeSong(&playing, &isPaused);
    resumeSong(&playing, &isPaused);  // should say "already playing"
```

```
46
47      // deleting a random song
48      std::cout << "\n--- Deleting a song ---\n";
49      deleteSong( title: "Caramel Pain", &head, &playing);
50      songList(head);
51
52      // deleting the "playing" song, this automatically stops the playing music (in pretend)
53      std::cout << "\n--- Deleting the currently playing song ---\n";
54      deleteSong( title: "If I Could Be A Constellation", &head, &playing); // this was "playing"
55      songList(head);
56
57      // playing is now nullptr
58      nextSong(&playing);
59
60      // now to delete the playlsit
61      std::cout << "\n--- Deleting entire playlist ---\n";
62      deleteplaylist(&head, &playing);
63      songList(head); // should report empty
64
65      return 0;
66  }
67
```

## OUTPUT

```
--- Default State ---
Playlist is empty.

Caramel Pain added.
Sparkling Daydream added.
Trapped in the Past added.
If I Could Be A Constellation added.

--- After adding songs ---
<-------------Start of Playlist------------->
1. New Genesis - Ado
2. Caramel Pain - Hoshimachi Suisei
3. Sparkling Daydream - ZAQ
4. Trapped in the Past - Tuyu
5. If I Could Be A Constellation - kessoku band
<-------------End of Playlist------------->

Now playing: Sparkling Daydream - ZAQ

--- Spamming Next Button ---

Now playing: Trapped in the Past - Tuyu

Now playing: If I Could Be A Constellation - kessoku band

Now playing: New Genesis - Ado

--- Clicking the Previous Button ---

Now playing: If I Could Be A Constellation - kessoku band

--- Pause and Resuming---
If I Could Be A Constellation - kessoku band is now paused...
Song is already paused
If I Could Be A Constellation - kessoku band is now resumed
Song is already playing

--- Deleting a song ---

 'Caramel Pain by Hoshimachi Suisei ' deleted.

<-------------Start of Playlist------------->
1. New Genesis - Ado
2. Sparkling Daydream - ZAQ
3. Trapped in the Past - Tuyu
4. If I Could Be A Constellation - kessoku band
<-------------End of Playlist------------->
```

```
--- Deleting the currently playing song ---

 'If I Could Be A Constellation by kessoku band ' deleted.

<-------------Start of Playlist------------->
1. New Genesis - Ado
2. Sparkling Daydream - ZAQ
3. Trapped in the Past - Tuyu
<-------------End of Playlist------------->

Nothing is playing...

--- Deleting entire playlist ---

Playlist successfully deleted.
Playlist is empty.


Process finished with exit code 0
```

## CONCLUSION

In this activity, I've learned a lot about the types of LINKED LISTS, but most especially how pointers work. Once you know how to visualize the processes in a linked list, it'll be much easier to understand.

Doing the procedure is like a learning journey. At first, I was just merely following how Sir Jim writes the code since I'm still "not in the know". After taking some time trying to understand it, I've got the bases down but still not to the molecular level. While writing down the code for the singly and doubly list, it became much clearer to me. What confused me at first was the "head", "second", "third", and so on pointer variables. I initially thought these were our nodes, but I was  off. These pointers only point to what we deem as the "head". It isn't the "head" itself. With this understanding, I came to the conclusion that we don't really need a "second", "third" and so on pointer variables as long as these 2 conditions are true; "we know what head and tail is" and "we cleanly link all the nodes in sequential order starting from the head to the tail". As long as these are met, we don't need to keep on initializing which is "second" or "third" we can just utilize functions to do that. So after coding the singly and doubly list, I found myself having fun writing the supplementary activity. With how I coded it, I wrote the traversal function similar to a singly list, but I kept the "prev" values so that we can go back to the previous songs. And everything else is just like a doubly list, BUT it loops back to the "head" which means, there'll be no pointers pointing to a null pointer and no tail, it will just be like a circular list.

In conclusion, I think I did a pretty decent job in this activity, I did prioritize my understanding. I'll need to improve on my error handling. With the code currently as it is, it'll only accept specific variables that meet the conditions I expect it to receive.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Explanation of Addresses and Pointers**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **It gave me a visualization of how the pointers behave and which addresses they are pointing so I can further understand how it works.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.